

GulfFlow Escrow – Submission Document

Builder: bentam

Contact: earlkeycsi400@gmail.com

Project: GulfFlow Escrow

Live Demo: <https://gulf-flow-escrow.vercel.app>

Source Code: <https://github.com/bincolase2/gulf-flow-escrow>

Network: Arc Testnet

Contract: 0x48Eebc62AFee8f7379a403E6cddC58b5F907b6a6

Deployment Proof:

<https://testnet.arcscan.app/tx/0x002395a127e7b0d92f842bfe51f6f40796cc0571bdd1b8466887131a728c6326>

1. Project Summary

GulfFlow Escrow is a hackathon-grade prototype for SME trade finance on Arc Testnet. It demonstrates how a buyer can price a supplier advance, connect a wallet, lock a trade deal into an escrow policy smart contract, move the deal through document approval, release, and completion states, then read the verified contract state back into the application.

The project targets cross-border SME trade flows, such as GCC buyers working with overseas suppliers. The goal is to reduce unsecured prepayment risk and provide transparent on-chain proof of the trade-finance lifecycle.

2. Problem

SMEs often need to pay suppliers before goods arrive or before documents are fully checked. This creates several problems:

- buyers risk paying before conditions are met;
- suppliers need confidence that payment will be released;
- document and milestone checks are often opaque;
- cross-border settlement can be slow and difficult to verify.

3. Solution

GulfFlow Escrow provides a simple escrow policy workflow:

- generate a demo trade-finance quote;
- connect an Arc Testnet wallet;
- lock the deal into a deployed smart contract;
- approve documents using a document hash;
- release the supplier advance state;
- complete or cancel the deal;
- read the final contract state back from Arc Testnet.

4. How the Demo Works

Step 1 – Price demo deal

The app creates a trade packet with invoice amount, supplier details, advance amount, risk score, required documents, and milestones.

Step 2 – Connect wallet

The user connects a browser wallet to Arc Testnet. The connected wallet acts as the buyer.

Step 3 – Lock demo deal

The user calls `lockDeal()` on the `GulfFlowEscrowPolicy` contract. The contract

records the buyer, supplier, invoice reference, invoice amount, advance amount, document hash, and Locked status.

Step 4 – Approve documents

The buyer calls `approveDocuments()`. The contract checks that the supplied document hash matches the locked deal and changes the status to `DocumentsApproved`.

Step 5 – Release advance

The buyer calls `releaseAdvance()`. The contract changes the status to `AdvanceReleased` and emits an on-chain event showing the supplier and advance amount.

Step 6 – Complete deal

The buyer calls `completeDeal()`. The contract moves the deal to `Completed`.

Step 7 – Read contract state

The UI calls `getDeal()` to read the stored data back from the contract and display the verified contract state.

5. Smart Contract Functions

The deployed contract is `GulfFlowEscrowPolicy`.

Main functions:

- `lockDeal(bytes32 dealId, address supplier, string invoiceRef, uint256 invoiceAmountUsdc6, uint256 advanceAmountUsdc6, bytes32 documentHash)`
- `approveDocuments(bytes32 dealId, bytes32 documentHash)`
- `releaseAdvance(bytes32 dealId)`
- `completeDeal(bytes32 dealId)`
- `cancelDeal(bytes32 dealId)`
- `getDeal(bytes32 dealId)`

6. What Is Live

The following parts are live on Arc Testnet:

- deployed `GulfFlowEscrowPolicy` contract;
- wallet connection to Arc Testnet;
- live contract writes from the UI;
- lock, approve, release, complete, cancel lifecycle actions;
- contract read-back via `getDeal()`;
- ArcScan links for transaction proof;
- full on-chain lifecycle test saved in the repository.

7. What Is Simplified

This is an honest hackathon-grade prototype, not a production escrow/custody product yet.

Simplified or future-production areas:

- document verification is represented by hashes;
- production KYB/KYC is not integrated;
- supplier acceptance is not yet a separate signature step;
- `releaseAdvance` updates escrow policy state and emits events, but does not custody or transfer real USDC yet;
- dispute handling and audited token custody would be required for production.

8. Repository Files of Interest

- contracts/GulfFlowEscrowPolicy.sol
- src/app/page.tsx
- src/lib/escrow-contract.ts
- src/lib/wallet.ts
- scripts/full-onchain-test.ts
- deployments/arc-testnet.json
- deployments/full-onchain-test.json
- DEVLOG.md
- docs/SUBMISSION.md

9. Full Lifecycle Proof

A full lifecycle test was executed on Arc Testnet and saved in:

deployments/full-onchain-test.json

The tested lifecycle includes:

- lockDeal
- approveDocuments
- releaseAdvance
- completeDeal

10. Submission Links

Live demo:

<https://gulf-flow-escrow.vercel.app>

GitHub repository:

<https://github.com/bincolase2/gulf-flow-escrow>

Contract address:

0x48Eebc62AFEE8f7379a403E6cddC58b5F907b6a6

Deployment transaction:

<https://testnet.arcscan.app/tx/>

0x002395a127e7b0d92f842bfe51f6f40796cc0571bdd1b8466887131a728c6326

11. One-Line Description

GulfFlow Escrow is an Arc Testnet dApp for SME trade finance that prices a demo supplier advance, locks the deal into an escrow policy contract, advances it through document approval and release states, and reads verified contract state back into the UI.